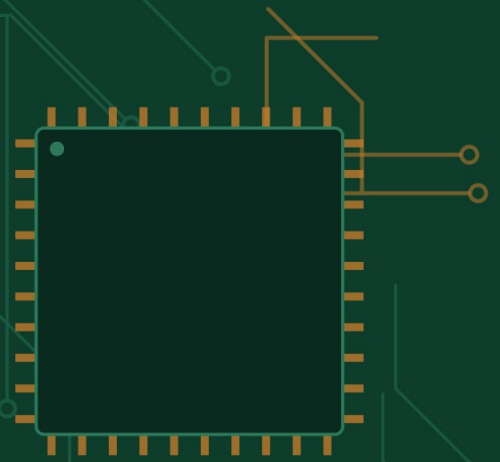
PERTEMUAN 12

Multitasking dengan FreeRTOS

Satu koki, banyak kompor: task, antrean, dan mutex di ESP32



Sistem Tertanam (Embedded Systems) · S1 Teknik Elektro

Dosen Pengampu: Anton Prafanto

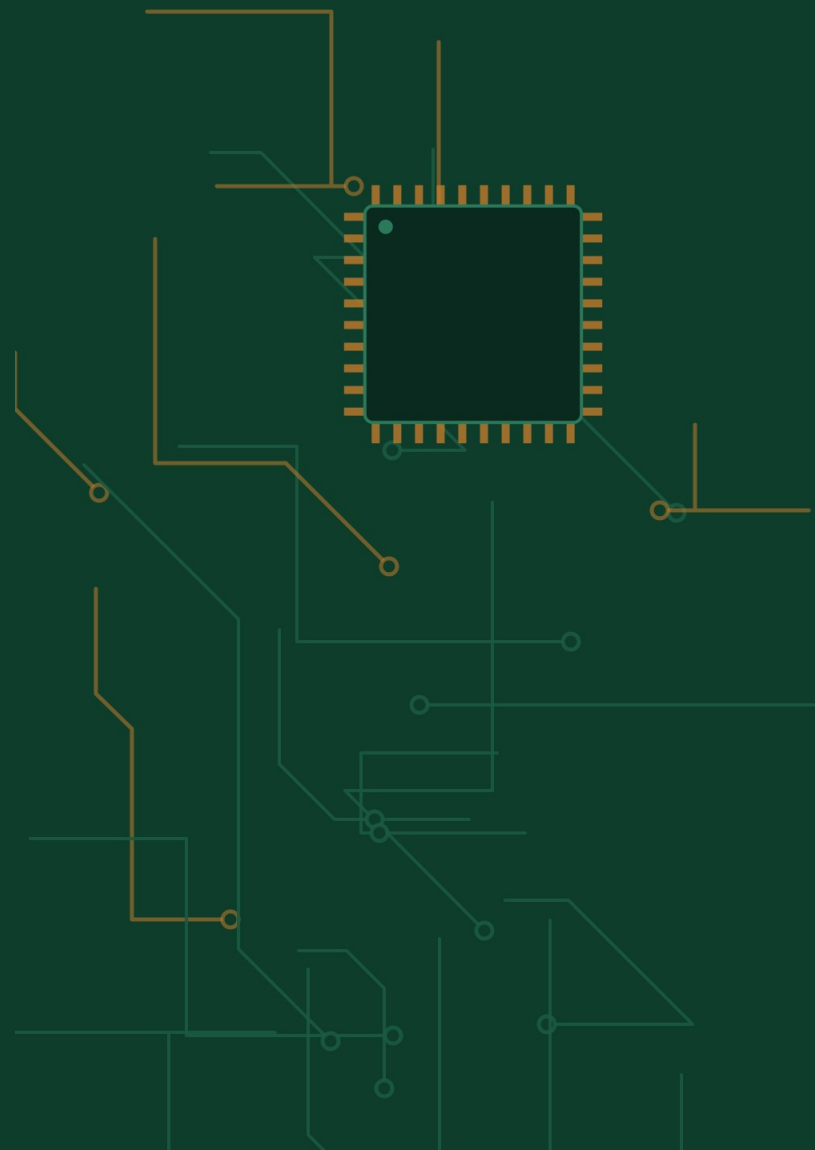
Tujuan Pembelajaran

- Membedakan **super-loop** dan **multitasking**, serta waktu nyata keras dan lunak
- Menjelaskan **task**, keadaan task, prioritas, dan penjadwal di ESP32 dua inti
- Membuat task dengan `xTaskCreatePinnedToCore()` dan memberi jeda yang benar
- Mengirim data dengan **antrean** dan melindungi sumber daya dengan **mutex**
- Mengenali kesalahan umum: stack kekecilan, watchdog, dan API di ISR

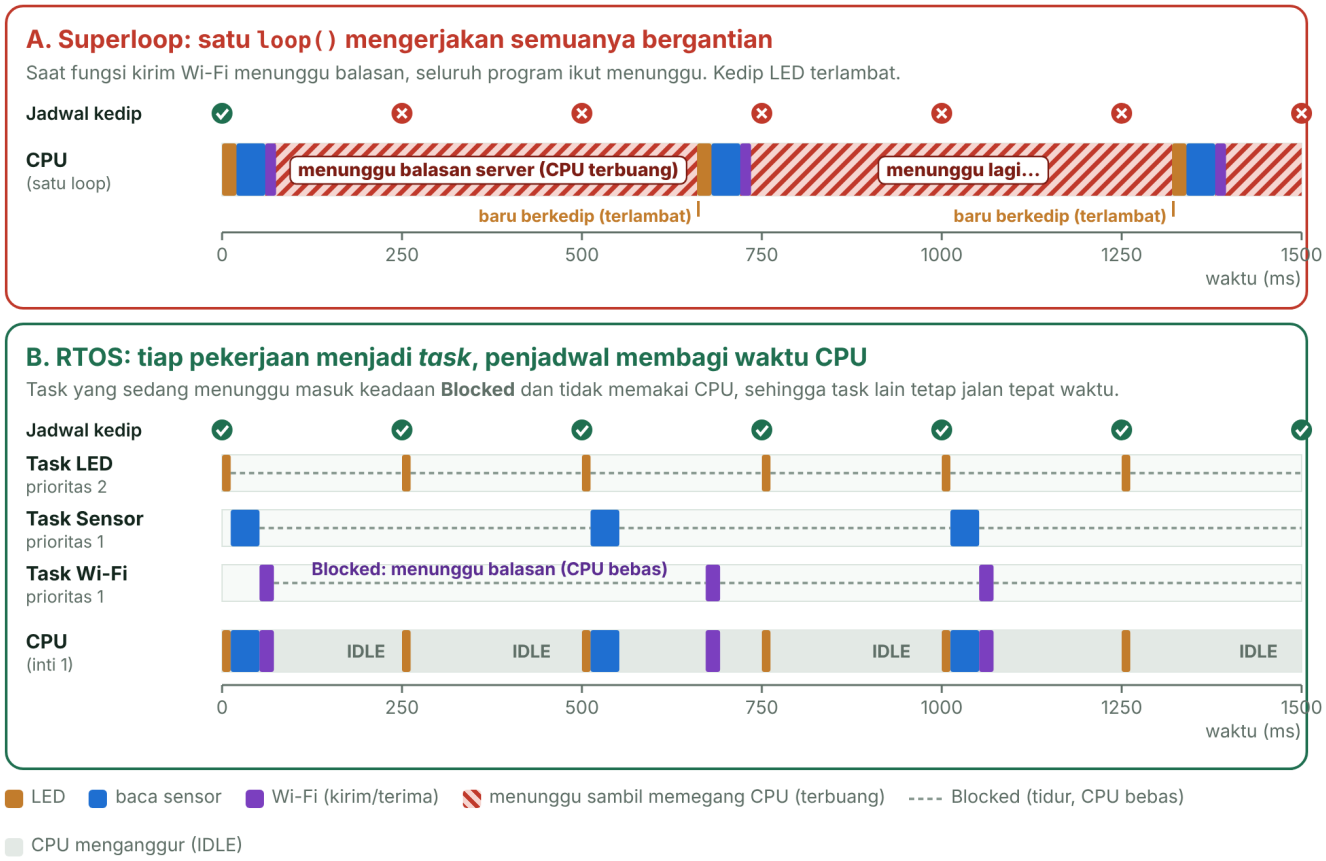
01

Mengapa Butuh RTOS?

Satu koki yang mengurus banyak kompor



Super-Loop vs RTOS



Task yang menunggu masuk keadaan Blocked, sehingga CPU bebas dipakai task lain.

Waktu Nyata: Lunak vs Keras



Soft real-time (lunak)

- Batas waktu terlewati → sistem terasa lambat
- Sistem masih bisa dipakai
- Contoh: ketikan keyboard telat muncul



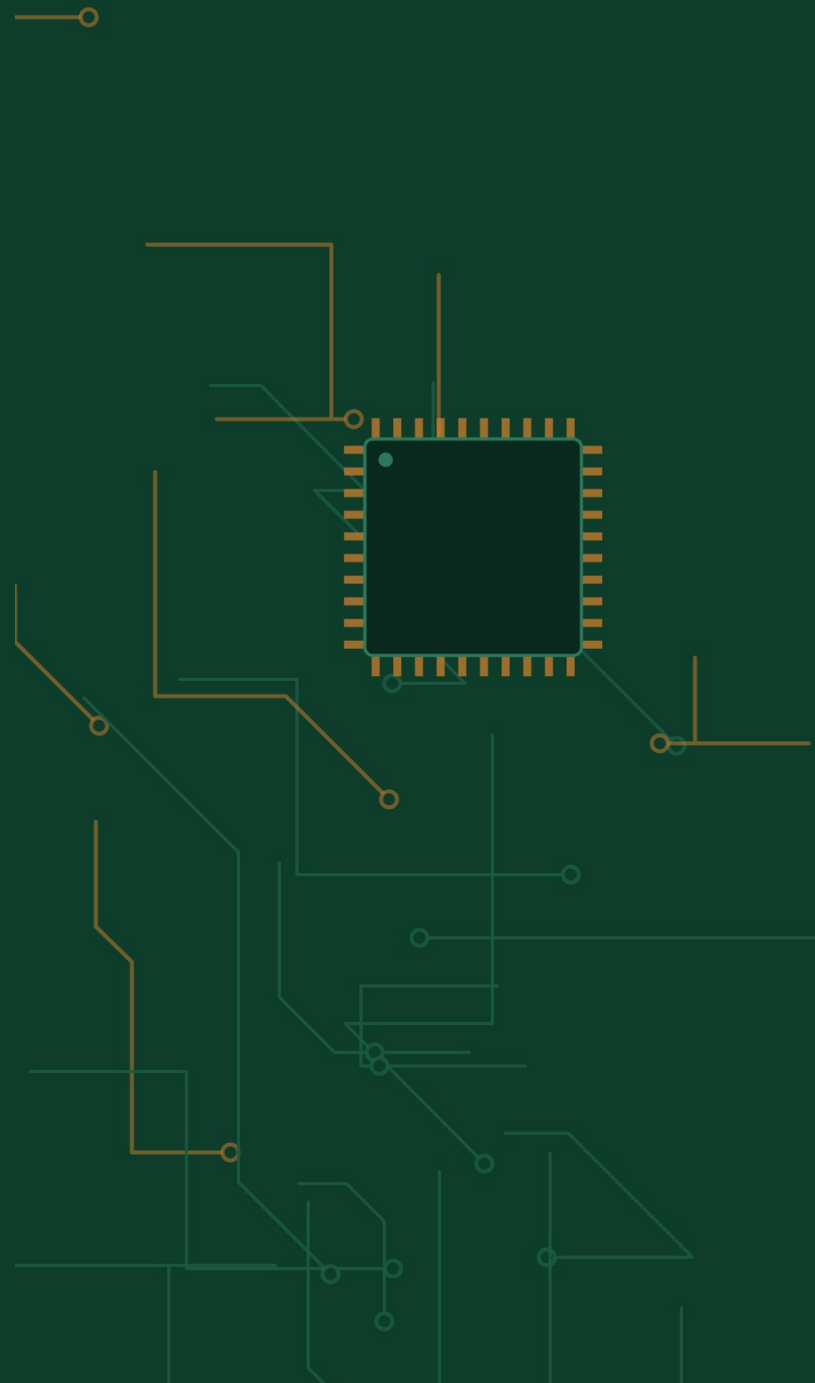
Hard real-time (keras)

- Batas waktu terlewati → **gagal total**
- Bisa membahayakan orang
- Contoh: airbag mobil terlambat mengembang

02

Task dan Penjadwal

Prioritas, preemption, tick, dan dua inti



Bentuk Sebuah Task

kode/01-dua-task/src/main.cpp

```cpp
void taskKedip(void *parameter) {
  for (;;) {                    // tanpa akhir
    digitalWrite(PIN_LED, HIGH);
    vTaskDelay(pdMS_TO_TICKS(500));
    digitalWrite(PIN_LED, LOW);
    vTaskDelay(pdMS_TO_TICKS(500));
  }
  // tidak boleh sampai ke sini
}
```

- Task = fungsi `void nama(void *)` berisi perulangan tanpa akhir
- Setiap task punya **stack** sendiri
- `vTaskDelay()` → task **Blocked**, CPU bebas
- `setup()` dan `loop()` pun dijalankan oleh sebuah task bernama `loopTask` (slide berikut)

Program Arduino-mu Juga Sebuah Task



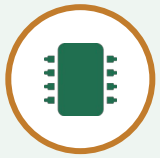
Stack 8 KB

Saat menyala, ESP32 membuat task `loopTask` yang memanggil `setup()` sekali, lalu `loop()` terus-menerus. Variabel lokalnya disimpan di *stack* **8.192 byte**. Larik lokal yang besar bisa membuatnya penuh (*stack overflow*).



Prioritas 1

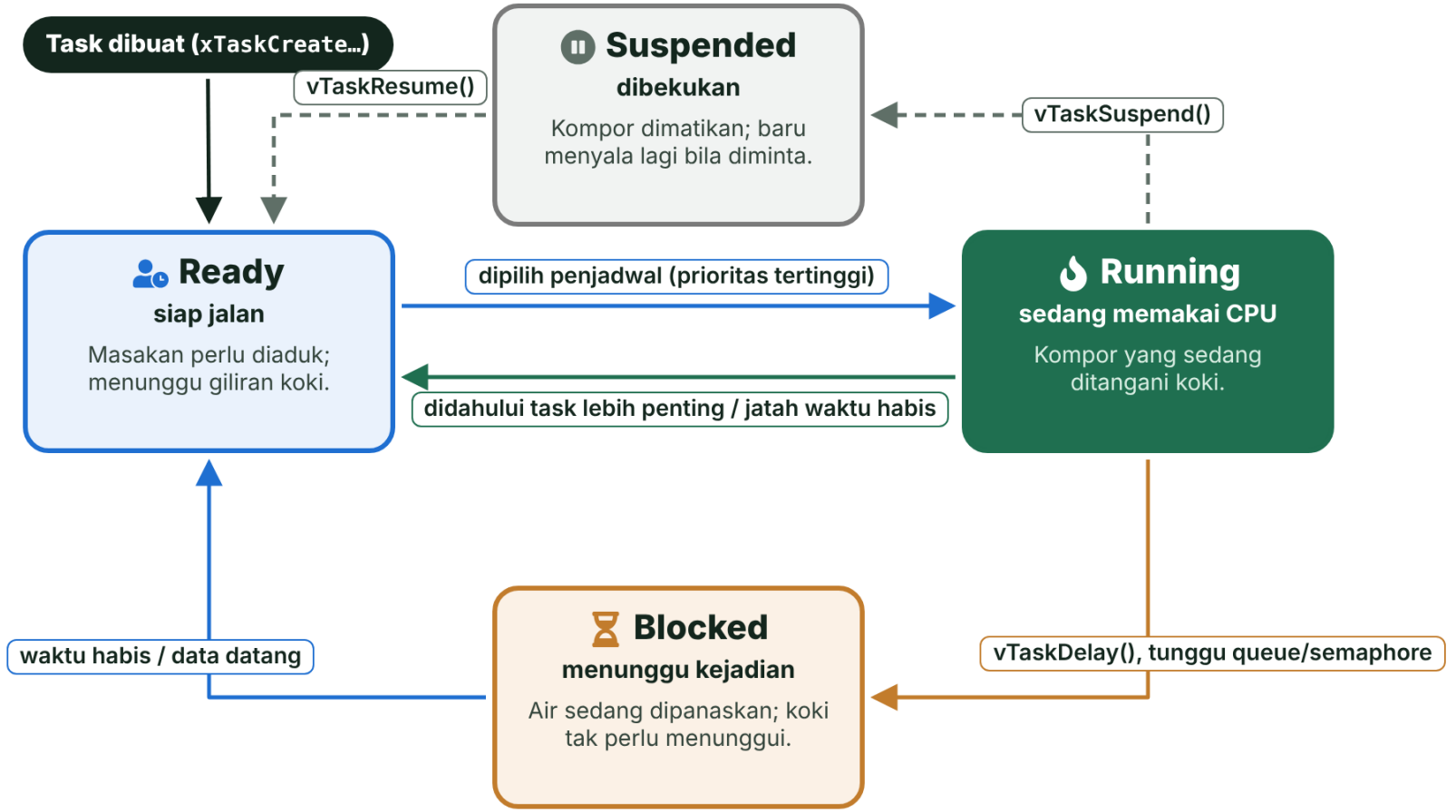
Prioritasnya rendah, hanya satu tingkat di atas task IDLE (0). Task buatanmu yang prioritasnya **lebih tinggi** di inti yang sama akan didahulukan.



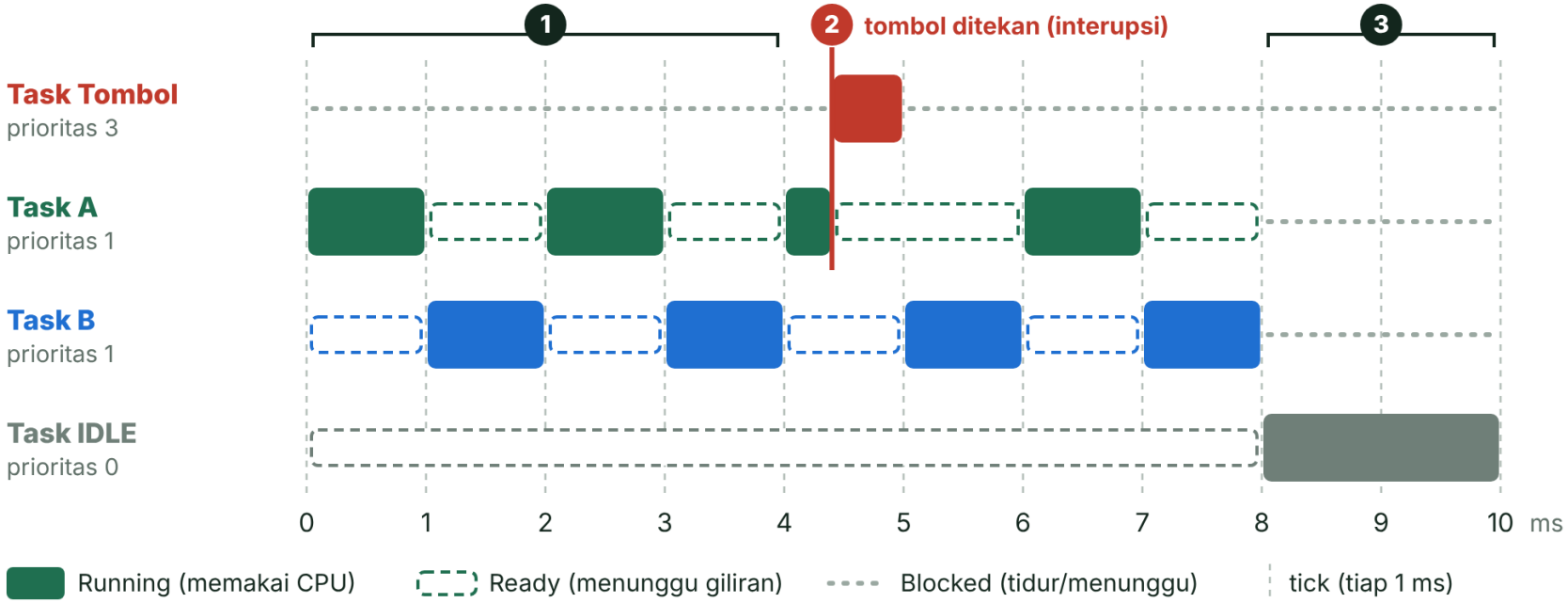
Inti 1

ESP32 punya dua inti. **Inti 0** umumnya mengurus Wi-Fi, sedangkan `loopTask` berjalan di **inti 1**. `delay()` di `loop()` sama dengan `vTaskDelay()`, jadi selama menunggu, CPU bisa dipakai task lain.

Siklus Hidup Task: Empat Keadaan



Prioritas, Preemption, dan Time Slicing



1 ***Time slicing***

Task A dan Task B sama-sama berprioritas 1, jadi keduanya bergantian memakai CPU setiap *tick*.

2 ***Preemption***

Tombol ditekan pada 4,4 ms. Task Tombol (prioritas 3) langsung merebut CPU dari Task A tanpa menunggu *tick* berikutnya.

3 **Task IDLE**

A dan B sedang tidur (`vTaskDelay`). Tidak ada task lain yang Ready, maka task IDLE (prioritas 0) yang berjalan.

ESP32 Punya Dua Inti

Inti 0 (PRO_CPU)

Bawaan untuk urusan protokol: Wi-Fi, Bluetooth, TCP/IP.

Wi-Fi driver radio (bila Wi-Fi dipakai)	tinggi
TCP/IP (lwIP) mengolah paket jaringan (bila Wi-Fi dipakai)	18
esp_timer layanan pewaktu sistem	22
taskKedip Praktikum 1 dibuat dengan argumen inti = 0	1
IDLE0 jalan bila tak ada task lain; **diawasi Task Watchdog**	0

Inti 1 (APP_CPU)

Bawaan untuk program aplikasi kita.

loopTask menjalankan `setup()` lalu `loop()` berulang; stack 8.192 byte	1
taskLapor Praktikum 1 dibuat dengan argumen inti = 1	1
arduino_events penyalur kejadian jaringan ke program (bila Wi-Fi dipakai)	19
IDLE1 jalan bila tak ada task lain di inti 1	0

Memori bersama (SRAM 520 KB): kedua inti melihat variabel yang sama. Karena itu data bersama perlu dijaga dengan **queue** atau **mutex**.

Anatomi xTaskCreatePinnedToCore()

```
xTaskCreatePinnedToCore(
  taskKedip,      // 1
  "Kedip",        // 2
  2048,           // 3
  NULL,           // 4
  1,              // 5
  &handleKedip,   // 6
  0);             // 7
```

```
// Isi task: perulangan tanpa akhir
void taskKedip(void *parameter) {
  for (;;) {
    digitalWrite(PIN_LED, HIGH);
    vTaskDelay(pdMS_TO_TICKS(500));
    digitalWrite(PIN_LED, LOW);
    vTaskDelay(pdMS_TO_TICKS(500));
  }
  // tidak boleh sampai ke sini (return)
}
```

1 Fungsi task
Nama fungsi yang dijalankan, berbentuk `void nama(void *parameter)`.

2 Nama task
Teks untuk *debugging*, misalnya muncul di pesan galat. Maks. 15 karakter.

3 Ukuran stack dalam BYTE
Di ESP-IDF satuannya **byte**, bukan *word* seperti FreeRTOS asli. Mulai dari 2048–4096.

4 Parameter
Penunjuk (*pointer*) ke data untuk task. Isi `NULL` bila tidak perlu.

5 Prioritas
0–24. Makin besar makin didahulukan. 0 dipakai task IDLE; `loop()` berprioritas 1.

6 Handle (pegangan)
Diisi alamat variabel `TaskHandle_t` untuk mengendalikan task nanti. Boleh `NULL`.

7 Inti (*core*)
`0`, `1`, atau `tskNO_AFFINITY` (bebas di inti mana saja).

Nilai kembali
`pdPASS` bila berhasil; gagal bila memori tidak cukup untuk stack.

Ukuran Stack: Byte, Bukan Word



Di ESP-IDF dan Arduino-ESP32, ukuran stack ditulis dalam **byte**. FreeRTOS asli memakai **word** (4 byte).

- Contoh internet `1000` = 4.000 byte di FreeRTOS asli, tetapi 1.000 byte di ESP32
- Mulai dari 2.048–4.096 byte, lalu ukur sisanya
- Nama task maks. 15 karakter; prioritas 0–24

Menunggu dengan Benar & Mengukur Stack

contoh

```
// Jeda biasa: 500 ms sejak dipanggil
vTaskDelay(pdMS_TO_TICKS(500));

// Periode tetap: bangun tiap 500 ms
TickType_t bangun = xTaskGetTickCount();
for (;;) {
  bacaSensor();
  xTaskDelayUntil(&bangun, pdMS_TO_TICKS(500));
}

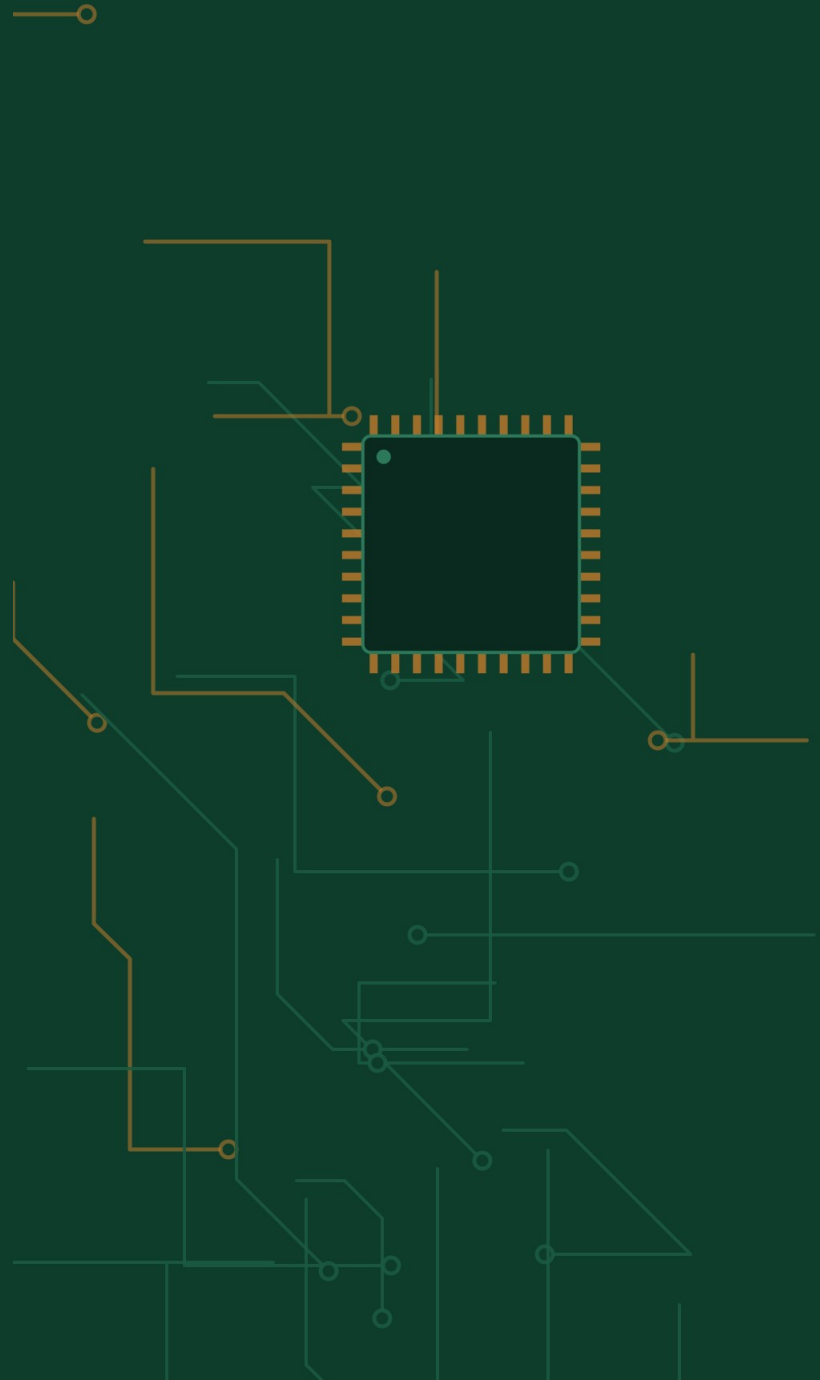
// Sisa stack paling sedikit (byte)
UBaseType_t sisa =
    uxTaskGetStackHighWaterMark(NULL);
```

- `delay()` di ESP32 = `vTaskDelay()`
- Busy-wait `while (millis() ...)` memonopoli CPU → hindari
- `xTaskDelayUntil()`: periode tidak bergeser
- Satu panggilan `Serial.printf()` saja bisa memakai ±900 byte *stack*

03

Antrean dan Mutex

Berbagi data dan sumber daya dengan aman



Antrean (Queue): Produsen–Konsumen



Masuk: `xQueueSend(q, &data, timeout)`

Bila antrean **penuh**: menunggu sampai *timeout*. Dengan timeout 0, fungsi langsung mengembalikan `errQUEUE_FULL` → data dibuang dan kita beri peringatan.

Keluar: `xQueueReceive(q, &data, timeout)`

Bila antrean **kosong**: task masuk keadaan **Blocked** dan tidak memakai CPU. Begitu data datang, task otomatis bangun.

Race Condition

Tanpa pelindung

	Task A (inti 0)	Task B (inti 1)	penghitung
1	baca → 5	—	5
2	—	baca → 5	5
3	tambah → 6	tambah → 6	5
4	tulis 6	—	6
5	—	tulis 6	6

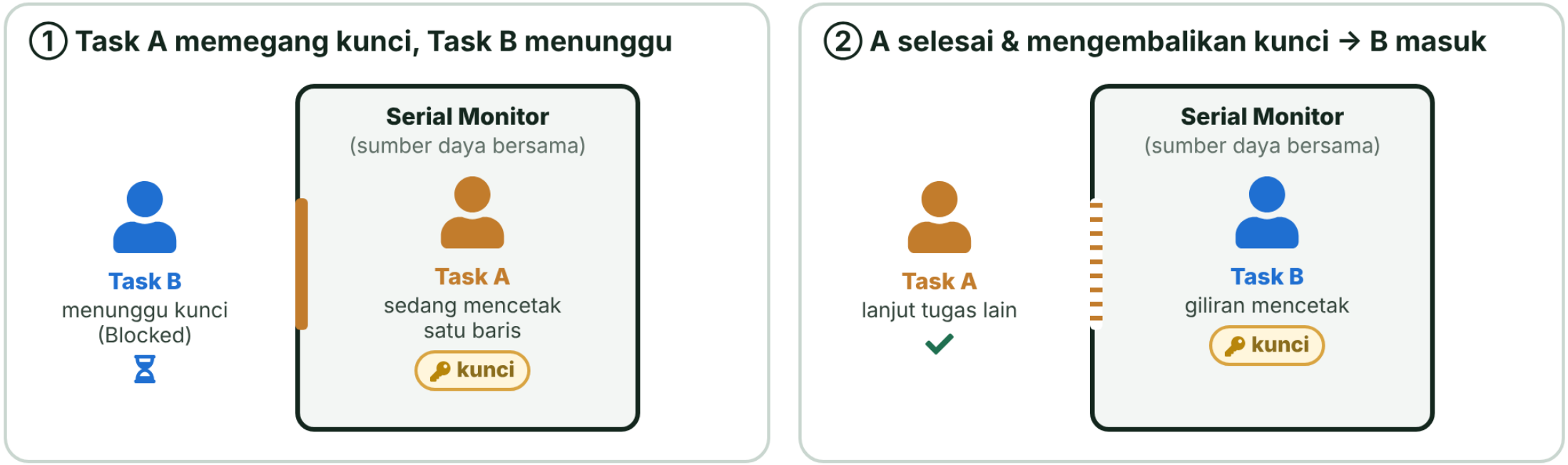
Ditambah dua kali, hasilnya 6, bukan 7. Satu tambahan hilang, dan kejadiannya bergantung pada waktu (sulit dilacak).

Dengan mutex (kunci)

	Task A (inti 0)	Task B (inti 1)	penghitung
1	ambil kunci ✓ baca → 5	ambil kunci → **menunggu**	5
2	tambah → 6 tulis 6	Blocked	6
3	kembalikan kunci	dapat kunci ✓	6
4	—	baca 6, tambah → 7 tulis 7	7
5	—	kembalikan kunci	7

Hasil selalu 7. Langkah baca–tambah–tulis dijalankan utuh oleh satu task sebelum task lain boleh masuk.

Mutex: Kunci Kamar Mandi



`kunci = xSemaphoreCreateMutex();`	Buat kunci sekali di `setup()`, sebelum task dibuat.
`xSemaphoreTake(kunci, portMAX_DELAY);`	Ambil kunci. Bila sedang dipakai, task menunggu (Blocked) tanpa memakai CPU.
`xSemaphoreGive(kunci);`	Kembalikan kunci **secepatnya**. Lupa mengembalikan = task lain menunggu selamanya.

Semaphore Biner vs Mutex

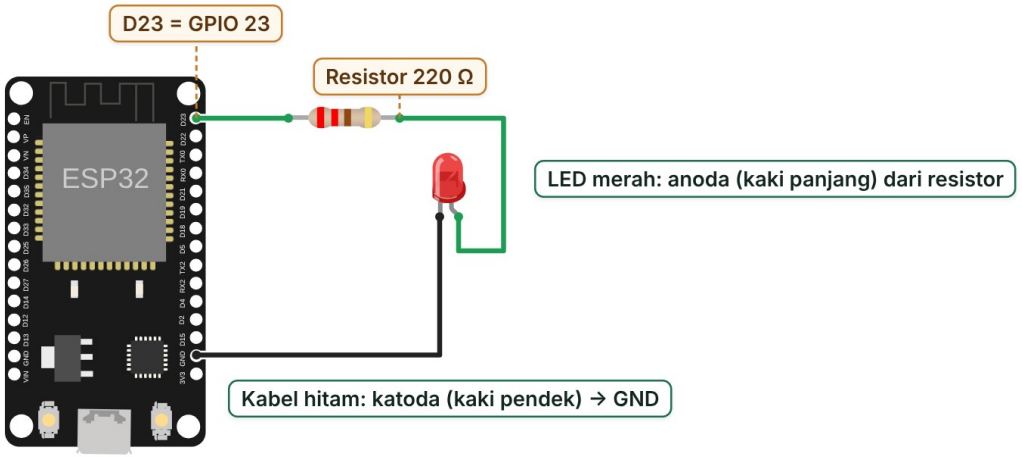
	Semaphore biner	Mutex
Kegunaan	Memberi sinyal (mis. ISR → task)	Mengunci sumber daya bersama
Yang mengembalikan	Boleh task atau ISR lain	Task pemegang kunci
Pewarisan prioritas	Tidak ada	Ada
Di dalam ISR	Boleh (xSemaphoreGiveFromISR)	Tidak boleh
Fungsi pembuat	xSemaphoreCreateBinary()	xSemaphoreCreateMutex()

Kesalahan yang Sering Terjadi

Gejala	Penyebab	Solusi
Restart: stack overflow in task …	Stack kekecilan	Perbesar, ukur sisa stack
Task watchdog got triggered (±5 s)	Busy-wait di inti 0	vTaskDelay() di tiap perulangan
Crash saat interupsi	API biasa dipanggil di ISR	Pakai versi …FromISR
Task prioritas rendah tak jalan	Prioritas tinggi tak pernah Blocked	Pastikan tiap task menunggu
Task should not return, Aborting now!	Fungsi task berakhir (return)	for (;;) atau vTaskDelete(NULL)

Praktikum 1: Dua Task di Dua Inti

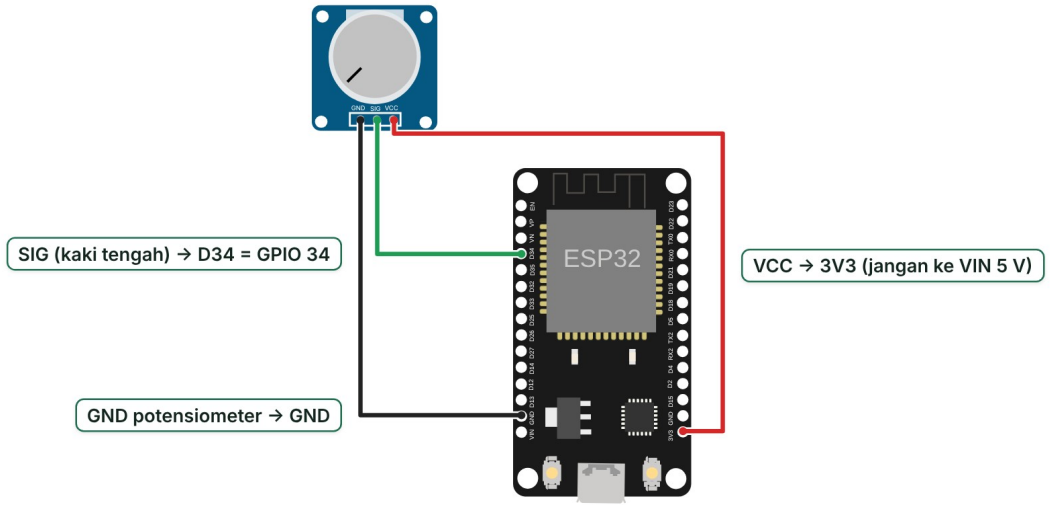
- LED + resistor 220 Ω di **GPIO 23**
- Task **Kedip** (inti 0): kedip tiap 0,5 s
- Task **Lapor** (inti 1): laporan tiap 2 s
- Hasil: `setup()` berjalan di `loopTask`, inti 1, prioritas 1
- Eksperimen: ganti jeda dengan busy-wait → watchdog



Gambar: @wokwi/elements (MIT)

Praktikum 2: Antrean Sensor ke Tampilan

- Potensiometer 10 kΩ di **GPIO 34**, VCC ke 3V3
- Task **Baca** (produsen, prioritas 2) tepat tiap 500 ms
- Task **Tampil** (konsumen) menunggu dengan `portMAX_DELAY`
- Eksperimen: konsumen lambat → antrean penuh



Gambar: @wokwi/elements (MIT)

Praktikum 3: Tanpa vs Dengan Mutex



PAKAI_MUTEX = false

- Dua task mencetak huruf demi huruf
- Contoh: `[A0[1] TaBs0k 1A ]m ...`
- Pola campuran berubah tiap kali



PAKAI_MUTEX = true

- Ambil kunci sebelum mencetak
- Setiap baris utuh
- Urutan baris A dan B boleh bertukar

Kuis Cepat



Task A (prioritas 1) sedang berjalan, lalu Task B (prioritas 3) di inti yang sama menjadi Ready. Apa yang terjadi?

A B menunggu sampai A selesai

B B langsung merebut CPU (preemption)

C A dan B bergantian setiap tick

D Watchdog me-restart ESP32

Rangkuman

- RTOS membagi pekerjaan menjadi **task** yang dijadwalkan penjadwal
- Keadaan task: Running, Ready, **Blocked**, Suspended
- Prioritas 0–24, preemption, time slicing, tick 1 ms, dua inti
- Stack dalam **byte**; ukur dengan `uxTaskGetStackHighWaterMark()`
- **Antrean** untuk kirim data, **mutex** untuk sumber daya bersama
- Di ISR pakai fungsi `...FromISR`

Sebelum Pertemuan 13



Tugas

- Latihan 1: tambah task ketiga
- Latihan 4: tombol dengan semaphore



Persiapan

- Baca modul Pertemuan 13: Wi-Fi dan MQTT
- Pikirkan: task mana yang mengirim data ke server?



Butuh Bantuan?

- Baca panduan/06-mengatasi-masalah.md
- Tanyakan di forum kelas